

Strategic Analysis of Food Delivery Logistics Performance Using Data Science

Week 1 Report: Strategic Planning and Data Exploration

Submitted by: Kashish
Internship Role: Logistics Data Analyst

1. Introduction

Delivery time is one of the most visible measures of service quality in online food delivery. Customers expect their orders to arrive quickly and predictably, while platforms must balance rider availability, vehicle capacity, traffic and cost. The World Bank's Logistics Performance Index frames logistics performance in terms of the speed and reliability with which goods move [1]. That index is designed for international trade, but the same ideas of speed and reliability apply directly to last-mile food delivery.

This report presents the Week 1 analysis completed during the Logistics Data Analyst internship. It uses exploratory data analysis (EDA), regression, K-Means clustering and a conceptual optimization framework to study a food-delivery dataset [2]. All numerical results in this report come from the accompanying Python notebook (Logistics_Delivery_Analysis.ipynb).

1.1 Research question

What factors influence food delivery time, and how can data-driven analysis help improve last-mile delivery efficiency?

2. Logistics Scenario

The scenario is last-mile food delivery in Indian cities. A customer places an order, a rider on a motorcycle, scooter or electric scooter collects it from the restaurant, and the rider delivers it to the customer. For every order, the dataset records the rider, the restaurant and delivery locations, weather and traffic conditions, vehicle type and condition, the number of additional deliveries on the trip, whether it was a festival day, the city category, and the total time taken.

The operations team needs answers to three practical questions: which conditions are associated with slow deliveries, how well delivery time can be predicted in advance, and which types of delivery should be prioritized for improvement.

3. Problem Statement

Delivery times in the dataset vary widely, from 10 to 54 minutes, with a mean of 26.58 minutes and a standard deviation of 9.34 minutes. Distance alone does not explain this variation. The problem addressed in this project is to identify which operational, environmental and rider-related factors are associated with delivery time, to build a model that predicts delivery time, to segment deliveries into meaningful groups, and to define a framework for reducing delay and cost.

4. Objectives

- Explore and clean the delivery dataset so that it is reliable for analysis.
- Compute baseline KPIs: delivery time, delivery distance and rider rating.

- Examine how traffic, weather, vehicle type, city category and multiple deliveries relate to delivery time.
- Build a simple regression model (distance only) and a multiple regression model (all selected factors), and compare them.
- Use K-Means clustering to identify operational delivery segments, especially high-delay ones.
- Define the objectives and constraints of a conceptual optimization framework.
- Review related literature and set out an end-to-end project roadmap.

5. Dataset Description

The analysis uses the publicly available Zomato food-delivery logistics dataset hosted on Hugging Face (allenborochin/zomato_delivery_EDA) [2]. After cleaning, the dataset contains 38,964 records and 22 columns, with no duplicate rows. The target variable is Time_taken (min). Table 1 groups the columns by type. The Python code used to load the data is shown in Listing 1 (Section 15).

Field group	Columns	Type
Identifiers	ID, Delivery_person_ID	Text
Rider profile	Delivery_person_Age, Delivery_person_Ratings	Numeric
Locations	Restaurant_latitude / longitude, Delivery_location_latitude / longitude	Numeric
Timing	Order_Date, Time_Orderd, Time_Order_picked	Text
Conditions	Weather_conditions, Road_traffic_density, Festival, City	Categorical
Order / vehicle	Type_of_order, Type_of_vehicle, Vehicle_condition, multiple_deliveries	Categorical / numeric
Target and derived	Time_taken (min), distance_km, delivery_speed	Numeric / categorical

Table 1: Dataset structure (38,964 records, 22 columns)

5.1 Summary statistics

Variable	Mean	Min	Median	Max
Delivery time (min)	26.58	10	26	54
Distance (km)	9.77	1.47	9.22	20.97
Rider rating	4.63	2.5	4.7	5.0
Rider age (years)	29.61	20	30	39
Multiple deliveries	0.75	0	1	3

Table 2: Summary statistics of key numeric variables

5.2 Composition of categorical variables

Category	Breakdown (number of deliveries)
City	Metropolitan 30,036 (77.1%); Urban 8,782 (22.5%); Semi-Urban 146 (0.4%)
Vehicle	Motorcycle 22,942 (58.9%); Scooter 12,926 (33.2%); Electric scooter 3,096 (7.9%)
Traffic	Low 13,139; Jam 12,414; Medium 9,556; High 3,855
Weather	Fog 6,660; Stormy 6,572; Cloudy 6,563; Sandstorms 6,489; Windy 6,436; Sunny 6,244
Order type	Snack 9,815; Meal 9,774; Drinks 9,751; Buffet 9,624
Festival	No 38,180; Yes 784 (2.0%)
Multiple deliveries	0: 12,165; 1: 24,704; 2: 1,782; 3: 313

Table 3: Distribution of categorical variables (labels as spelled in the dataset)

6. Key Performance Indicators (KPIs)

Table 4 lists the baseline KPIs computed in the notebook. They are performance measures of the delivery operation and are not model accuracy figures. Any future improvement should be measured against them. The calculation code is shown in Listing 3 (Section 15).

KPI	Value	Why it matters
Average delivery time	26.58 minutes	Primary service-level measure
Average delivery distance	9.77 km	Drives rider time and cost
Average delivery-person rating	4.63 / 5	Indicator of service quality

Table 4: Baseline KPIs from the notebook

An on-time delivery rate was not calculated as a KPI. The dataset contains no promised delivery time and no explicit on-time indicator, so a delivery cannot be classified as on time or late. If a promised delivery time is recorded in future data, on-time delivery rate should be added to this KPI set.

7. Data Cleaning

The data was checked for missing values and duplicates before modelling. Table 5 summarizes the findings and treatment. The code is shown in Listing 2 (Section 15).

Column	Missing values	Share of rows	Treatment
Delivery_person_Age	1,019	2.6%	Filled with median (30.0)
Delivery_person_Ratings	1,055	2.7%	Filled with median (4.7)
Time_Orderd	835	2.1%	Not required for the initial regression analysis
Duplicate rows	0	0%	None required

Table 5: Missing values and treatment

Median imputation was appropriate for the two numeric variables for three reasons. First, the median is not distorted by extreme values, unlike the mean. Second, it keeps the centre of each distribution almost unchanged. Third, only about 2.6% to 2.7% of rows were affected, so the imputation has little influence on the overall results. After imputation, both of these columns had no missing values. Missing values were not removed from the whole dataset: Time_Orderd still contains 835 missing entries.

Time_Orderd contained some missing values, but it was not required for the initial regression analysis and was therefore not used in the models. It would need to be parsed and treated if time-of-day analysis is added in future work. Delivery time itself had no missing or impossible values. For regression, categorical variables were converted using one-hot encoding, which expanded the 11 selected input features into 21 encoded features.

8. Exploratory Data Analysis (EDA)

This section presents the notebook's exploratory charts. Taken together, they show that delivery performance is associated with several operational factors and not with distance alone.

8.1 Delivery time distribution

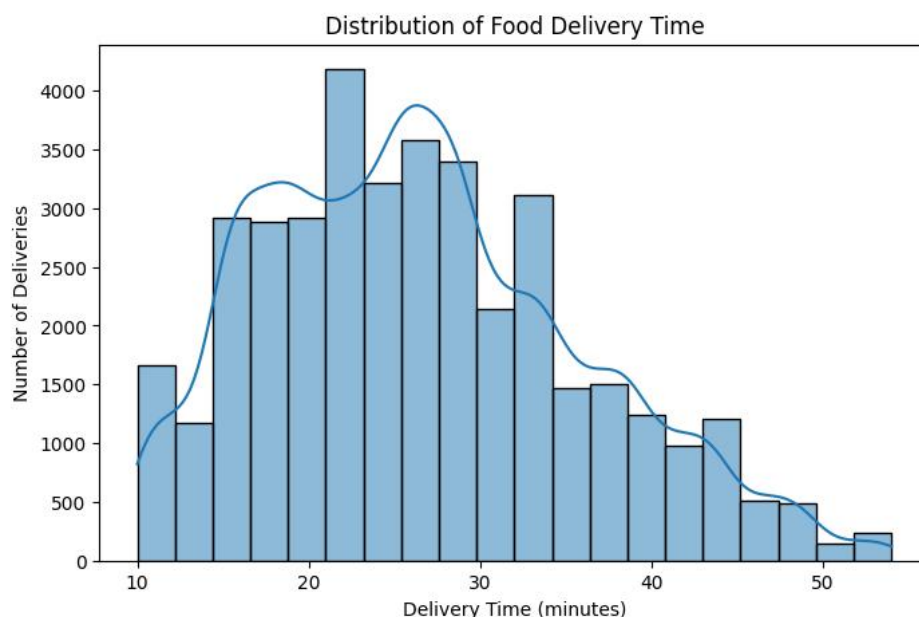


Figure 1: Distribution of Food Delivery Times (minutes)

Delivery times range from 10 to 54 minutes and are centred near 26 minutes. Half of all orders fall between 19 and 33 minutes, so a typical customer waits about half an hour. A noticeable group of orders takes 40 minutes or more.

8.2 Distance versus delivery time

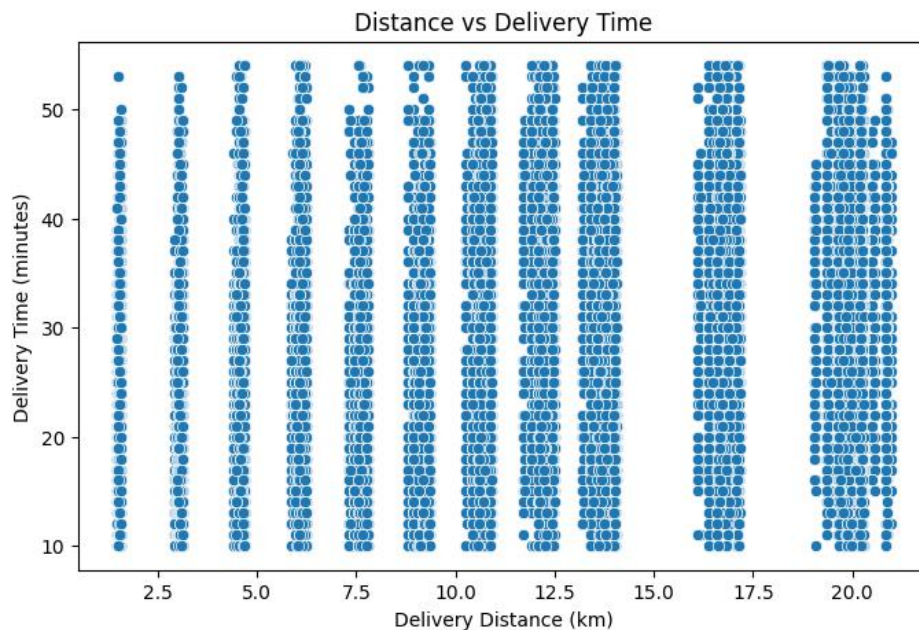


Figure 2: Distance vs Delivery Time

Longer distances tend to be associated with longer delivery times, but the scatter is very wide at every distance. A 10 km delivery can take anywhere from about 10 to over 50 minutes. This is an early indication that distance is only one of several relevant factors.

8.3 Road traffic density

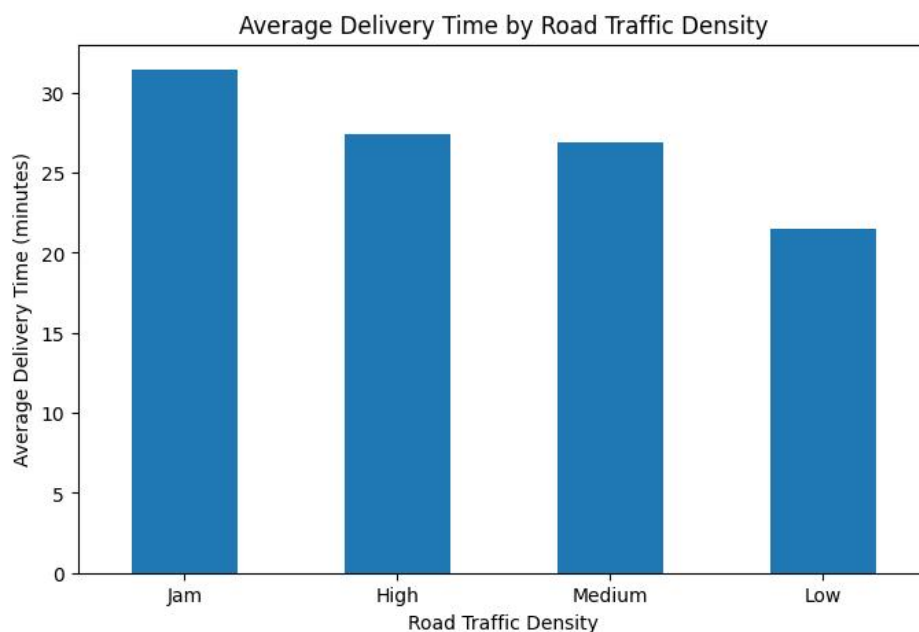


Figure 3: Average delivery time by road traffic density

Traffic density	Avg. time (min)
Jam	31.44
High	27.41
Medium	26.93
Low	21.50

Table 6: Average delivery time by traffic density

Jam conditions are associated with an average delivery time almost 10 minutes (about 46%) higher than low traffic. Jam and High traffic together cover 16,269 deliveries, roughly 42% of all orders, so this pattern affects a large share of the operation.

8.4 Vehicle type

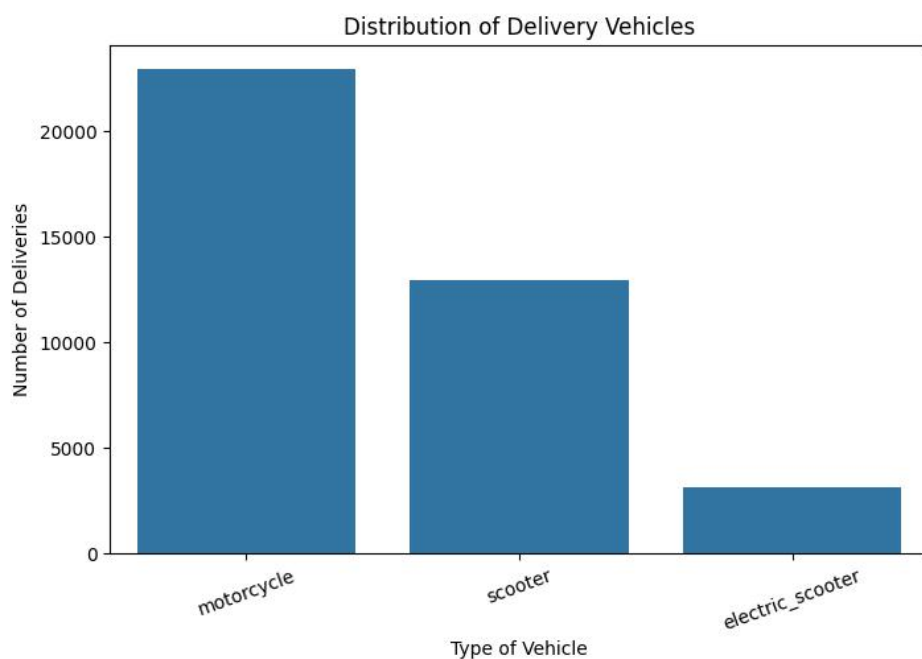


Figure 4: Number of deliveries by vehicle type

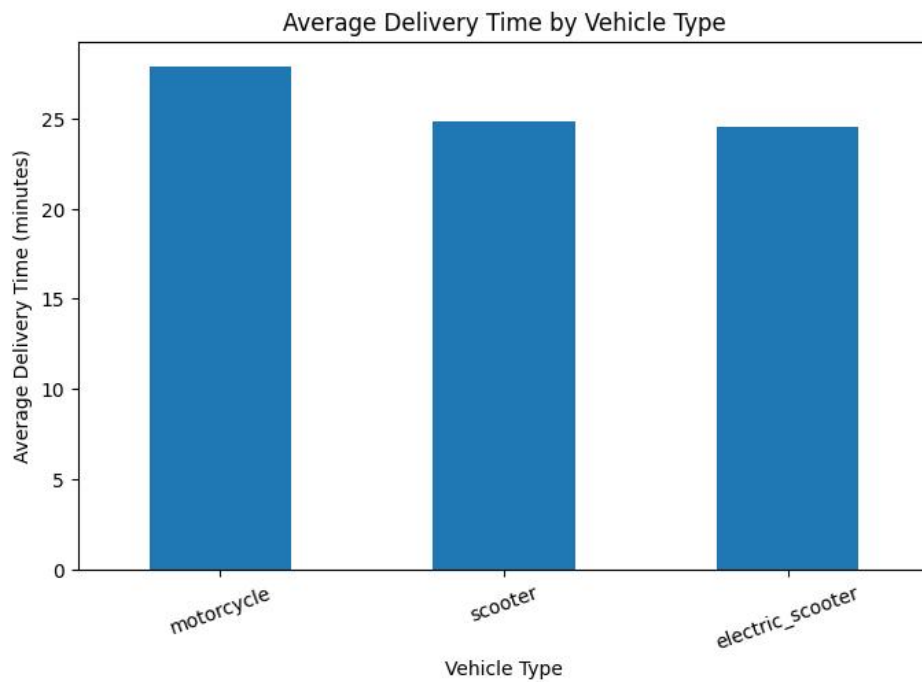


Figure 5: Average delivery time by vehicle type

Motorcycles handle nearly 59% of deliveries and have the longest average time (27.85 minutes), compared with 24.81 minutes for scooters and 24.56 minutes for electric scooters. This difference should be read with care because vehicle type may be linked to city or distance, which the EDA alone cannot separate.

8.5 City category

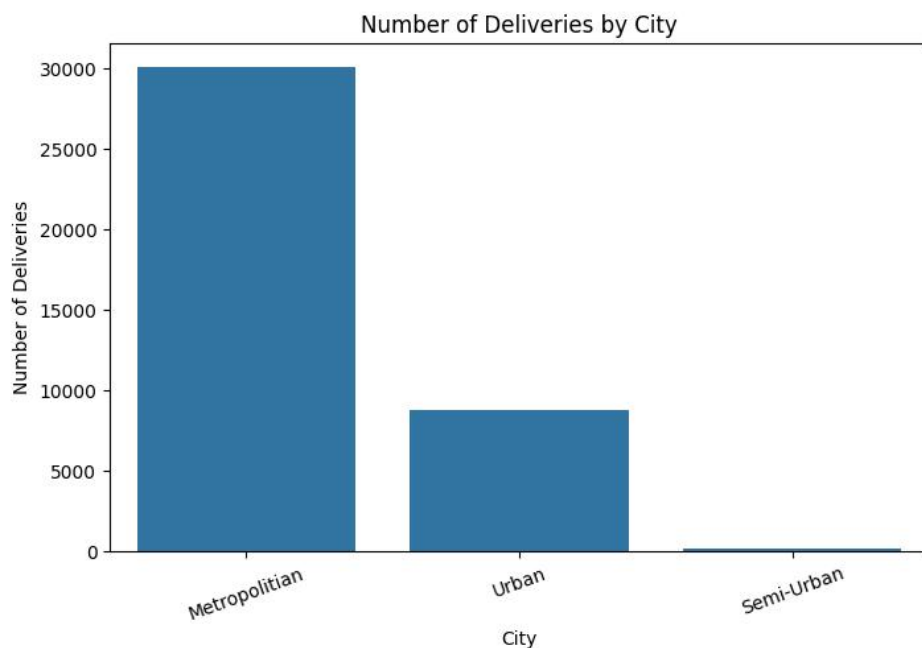


Figure 6: Number of deliveries by city category

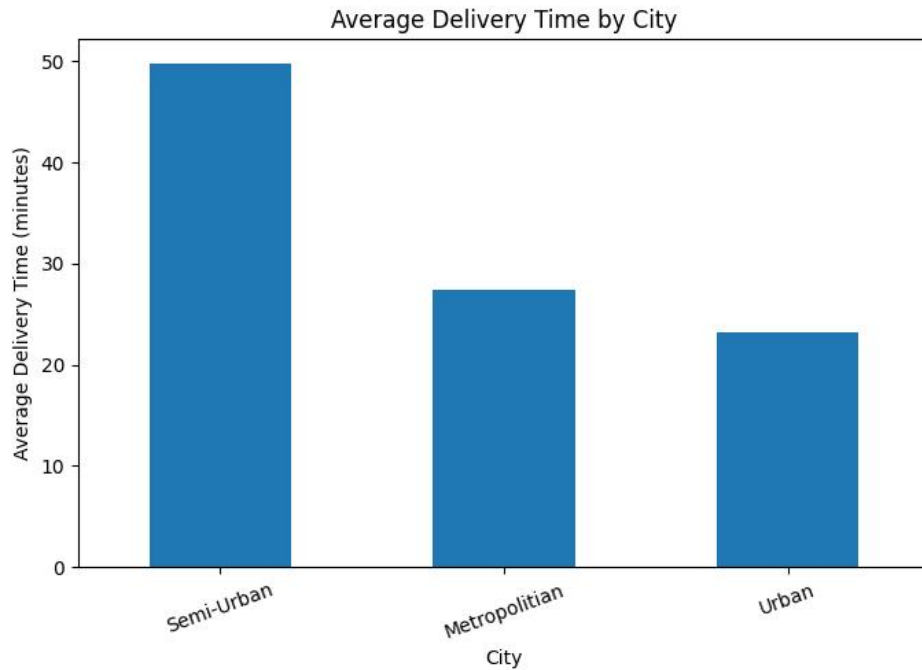


Figure 7: Average delivery time by city category

Metropolitan cities account for 77% of deliveries. Semi-Urban deliveries have a much higher average time (49.69 minutes) than Metropolitan (27.45) and Urban (23.24) deliveries. However, the Semi-Urban category contains only 146 records, so this result should be interpreted cautiously.

8.6 Weather conditions

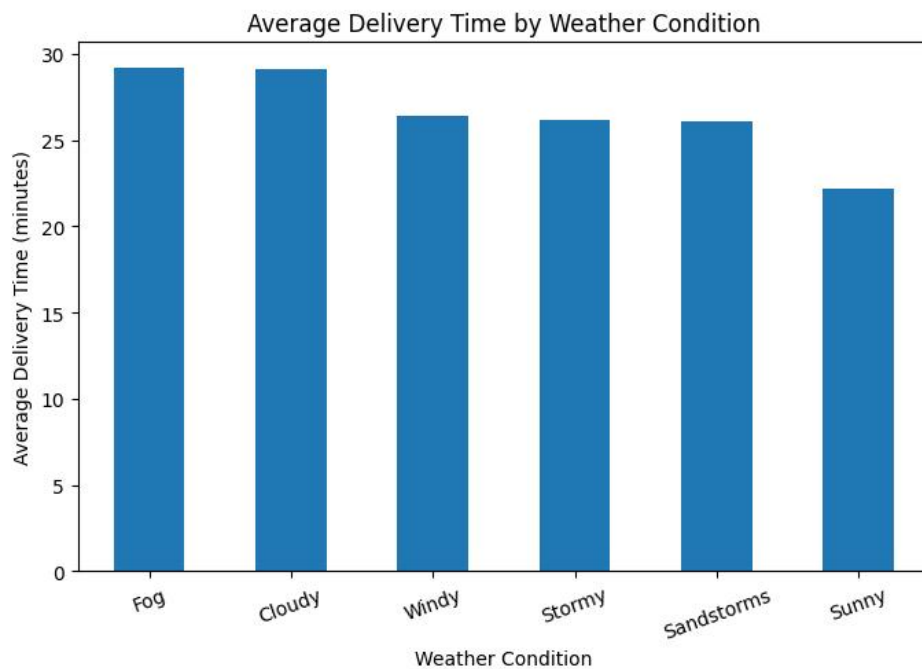


Figure 8: Average delivery time by weather condition

Sunny conditions show the lowest average delivery time (22.20 minutes). Fog (29.22) and Cloudy (29.15) show the highest, about 7 minutes above Sunny. Windy, Stormy and Sandstorms conditions sit in the middle at roughly 26 minutes.

8.7 Multiple deliveries

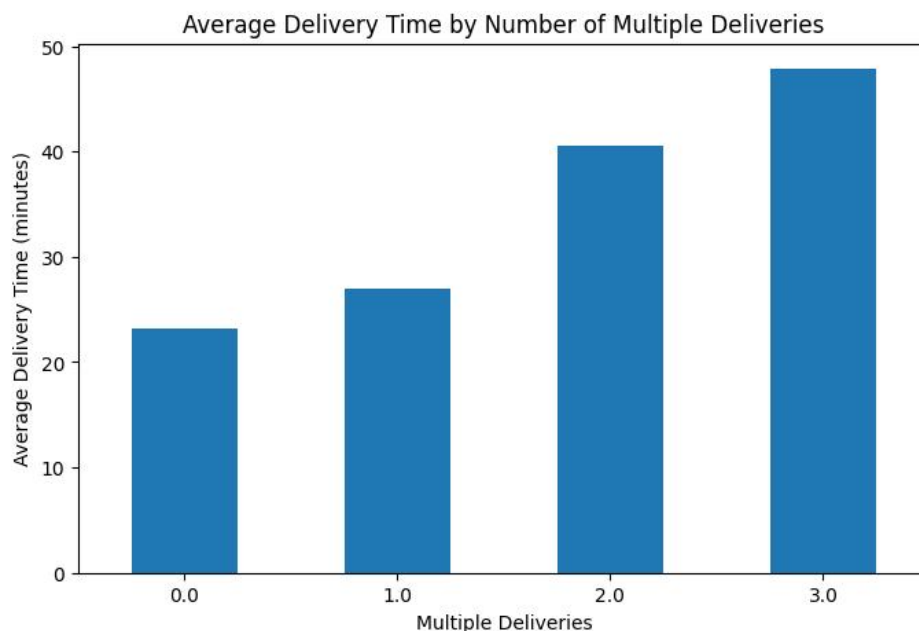


Figure 9: Average delivery time by number of multiple deliveries

Parallel deliveries	Orders	Avg. time (min)
0	12,165	23.16
1	24,704	27.00
2	1,782	40.49
3	313	47.85

Table 7: Average delivery time by number of parallel deliveries

Average delivery time rises sharply with the number of parallel deliveries, from 23.16 minutes with none to 47.85 minutes with three. Most riders carry 0 or 1 additional orders, but the small group carrying 2 or 3 shows much longer times. This suggests that rider workload is an important operational factor to monitor.

9. Simple Linear Regression

A simple linear regression was fitted with `distance_km` as the only predictor and `Time_taken (min)` as the target, using an 80/20 train-test split (`random_state = 42`). The model was implemented with `scikit-learn` [5]; the code is shown in Listings 8 and 9 (Section 15).

Metric	Result
R ² score	0.1084
MAE	7.07 minutes
RMSE	8.71 minutes

Table 8: Simple linear regression results on the test set

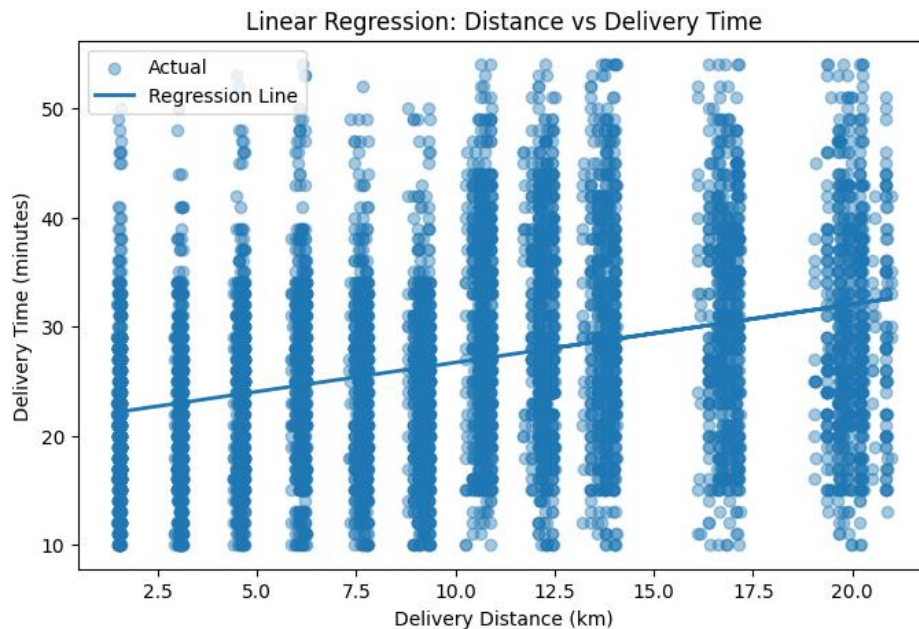


Figure 10: Simple linear regression: distance (km) versus delivery time (minutes)

R² measures the share of variation in delivery time explained by the model. It is not a measure of accuracy. An R² of 0.1084 means that distance explains only about 10.8% of the variation in delivery time. The average prediction error (MAE) is 7.07 minutes, which is large relative to the 26.58 minute average delivery time. Distance alone therefore has limited explanatory power.

10. Multiple Linear Regression

The multiple regression model used 11 features: distance_km, Delivery_person_Age, Delivery_person_Ratings, Vehicle_condition, multiple_deliveries, Weather_conditions, Road_traffic_density, Type_of_order, Type_of_vehicle, Festival and City. Categorical variables were converted using one-hot encoding, giving 21 encoded features. The same 80/20 split was used so the two models can be compared fairly. The code is shown in Listings 10 to 13 (Section 15).

Model	R ²	MAE	RMSE
Simple Linear Regression	0.1084	7.07 min	8.71 min
Multiple Linear Regression	0.5903	4.69 min	5.90 min

Table 9: Comparison of simple and multiple linear regression results (test set)

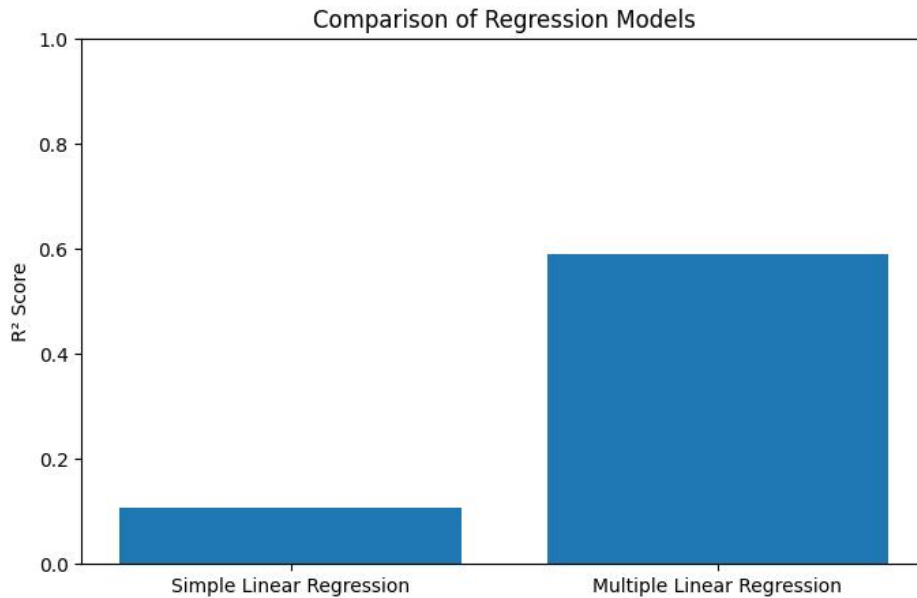


Figure 11: Regression Model Comparison: R^2 of the simple and multiple linear regression models

The multiple regression model performs substantially better than the distance-only model because it incorporates several operational factors. It explains about 59% of the variation in delivery time, and MAE falls from 7.07 to 4.69 minutes (RMSE from 8.71 to 5.90 minutes). The comparison code is shown in Listing 14 (Section 15). About 41% of the variation remains unexplained, which is expected because variables such as time of day and restaurant preparation time are not yet included.

10.1 Largest regression coefficients

Coefficients for categorical variables represent differences in minutes relative to the reference (dropped) category: High traffic, Cloudy weather, Buffet orders, electric scooter, no festival and Metropolitan city. They show associations in the data and do not prove that a factor causes delay.

Feature	Coefficient (min)	Interpretation
City_Semi-Urban	+9.93	Associated with longer time than Metropolitan
Festival_Yes	+9.40	Festival days associated with about 9 more minutes
Delivery_person_Ratings	-7.46	Higher-rated riders associated with shorter time
Road_traffic_density_Low	-6.61	Low traffic associated with shorter time than High
Weather_conditions_Sunny	-6.15	Sunny associated with shorter time than Cloudy
multiple_deliveries	+2.89	Each extra delivery associated with about 2.9 more minutes
Weather_conditions_Stormy	-2.85	Shorter than Cloudy
Weather_conditions_Sandstorms	-2.81	Shorter than Cloudy
Weather_conditions_Windy	-2.72	Shorter than Cloudy
Road_traffic_density_Medium	-2.47	Shorter than High traffic
Vehicle_condition	-2.22	Higher condition score associated with shorter time
City_Urban	-1.93	Shorter than Metropolitan

Table 10: Largest coefficients from the multiple regression model

Three cautions apply. First, the Semi-Urban category contains only about 146 observations, so its coefficient should be interpreted cautiously; the Festival group is also small (784 orders). Second, rider rating may partly reflect speed itself, since faster riders may receive better ratings, so it shows an association rather than a cause. Third, the Jam coefficient is only +0.51 relative to High traffic, which means that once other factors are included, Jam and High traffic behave similarly. The large gap in Section 8.3 is mainly the difference between congested traffic and Low traffic.

11. K-Means Clustering

K-Means clustering was performed on three features: distance_km, Time_taken (min) and Delivery_person_Ratings. The features were standardized using StandardScaler, and the elbow method was used to evaluate K from 2 to 7. The code is shown in Listings 15 to 18 (Section 15).

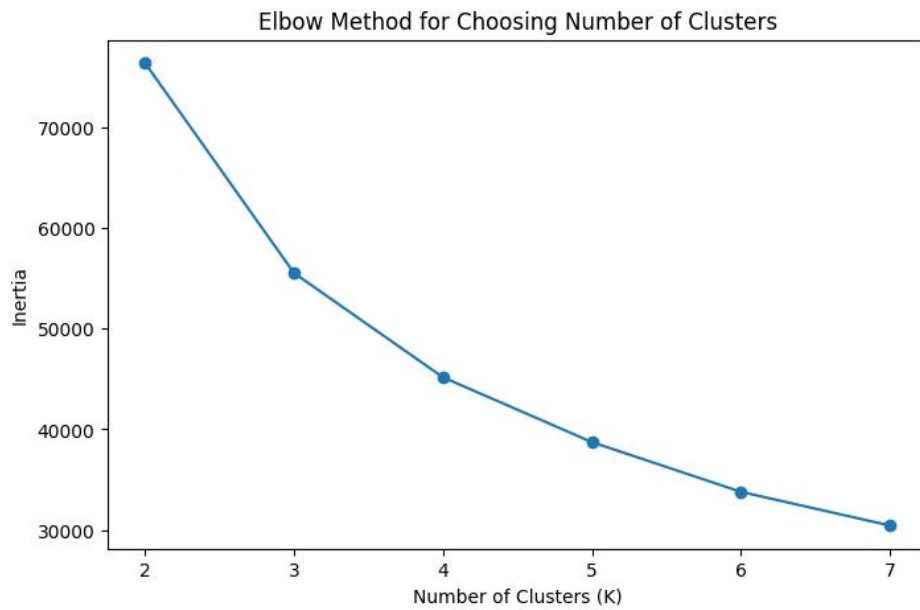


Figure 12: Elbow method: inertia for K = 2 to 7

Inertia falls steadily, from about 76,000 at K = 2 to about 30,000 at K = 7, without a sharp elbow. K = 5 was selected as a practical balance between detail and interpretability (random_state = 42, n_init = 10). The clustering was done with scikit-learn [5].

Cluster	Deliveries	Share	Avg. distance (km)	Avg. time (min)	Avg. rating
0	9,995	25.7%	6.14	28.03	4.72
1	4,892	12.6%	14.82	40.89	4.69
2	10,045	25.8%	5.07	16.80	4.74
3	8,858	22.7%	15.42	22.51	4.75
4	5,174	13.3%	11.43	36.22	4.01

Table 11: K-Means cluster profile (cluster averages)

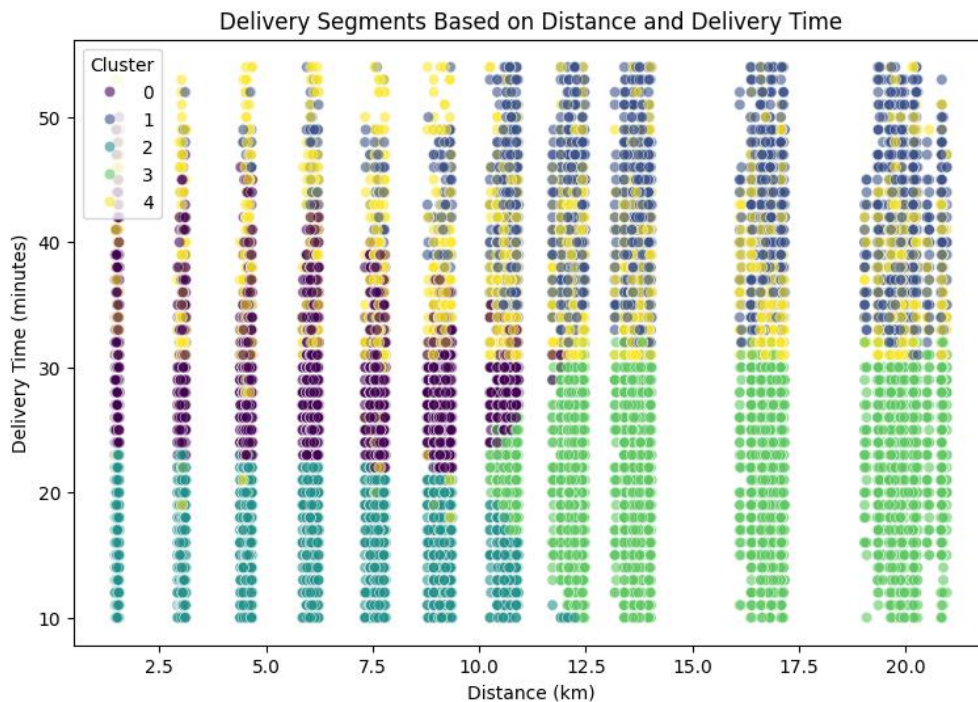


Figure 13: K-Means Clustering: delivery segments ($K = 5$) by distance and delivery time

11.1 Segment interpretation

- **Cluster 0:** short-distance deliveries with moderate delivery time (6.14 km, 28.03 minutes).
- **Cluster 1:** long-distance deliveries with the highest average delivery time (14.82 km, 40.89 minutes).
- **Cluster 2:** short-distance deliveries and the fastest segment (5.07 km, 16.80 minutes).
- **Cluster 3:** the longest-distance deliveries but relatively fast performance (15.42 km, 22.51 minutes).
- **Cluster 4:** medium-distance, slower deliveries with the lowest average rating (11.43 km, 36.22 minutes, rating 4.01).

The key insight is that distance alone does not determine delivery time. Cluster 3 has an average distance of 15.42 km but an average delivery time of only 22.51 minutes, while Cluster 0 has a much shorter average distance of 6.14 km but an average delivery time of 28.03 minutes. This suggests that other operational factors, such as traffic, workload, weather, rider characteristics or delivery conditions, may influence performance. Clusters 1 and 4 together make up about 26% of deliveries and are the most obvious candidates for operational attention. The colours in the scatter plot overlap because three features are shown in two dimensions, so the averages in Table 11 are a more reliable guide than the chart.

12. Optimization

This project presents a conceptual optimization framework. It does not implement a route-optimization algorithm. Its purpose is to define what should be optimized and under which

limits, using the findings of Sections 8 to 11 as inputs. The Python code that defines the objectives and constraints is shown in Listing 19 (Section 15).

12.1 Objectives

- Minimize delivery time.
- Reduce transportation cost.
- Improve rider and resource utilization.
- Prioritize high-delay delivery segments.

12.2 Constraints

- Limited number of riders.
- Vehicle availability.
- Delivery distance.
- Traffic conditions.
- Delivery capacity.

12.3 Link to the analysis and future extensions

The analysis indicates where optimization effort could be focused: riders carrying several parallel deliveries (Table 7), orders placed in High or Jam traffic (about 42% of deliveries), and the slow segments Clusters 1 and 4. The multiple regression model could supply estimated delivery times for such a framework.

Future versions could formalize the framework as a Vehicle Routing Problem (VRP) with capacity constraints and time windows, add route optimization, use dynamic order dispatching, and bring in real-time traffic information. None of these methods was implemented in this project. The literature on last-mile VRPs [3] and on meal-delivery dispatching and routing [4] provides starting points for these extensions.

13. Literature Research

Four strands of literature support this project.

- **Logistics performance.** The World Bank's Logistics Performance Index describes logistics performance in terms of speed and reliability and highlights widespread differences in delays and reliability across the world [1]. It addresses international trade, so it is used here as context rather than as a direct benchmark for food delivery.
- **Last-mile routing.** Jazemi et al. review the literature on vehicle routing for last-mile delivery in urban areas and discuss aspects such as fleet capacity, time windows and dynamic input [3]. These match the capacity and time-window constraints proposed for future versions of the optimization framework.
- **Food-delivery optimization.** Auad et al. study order-courier assignment and routing for meal delivery using real operational data from Meituan, and report that the largest

improvements occur during high-demand periods with courier shortages [4]. This supports the focus on rider availability and workload in this report.

- **Tools.** Regression and K-Means were implemented with scikit-learn [5].

Together, these sources support the approach followed here: describe and clean the data, predict delivery time, segment the operation, and then move towards routing and dispatch optimization.

14. End-to-End Project Roadmap

Step	Stage	Description	Status
1	Data collection	Obtain the Hugging Face Zomato delivery dataset [2]	Completed
2	Data cleaning	Check missing values and duplicates; median imputation for age and rating	Completed
3	Feature engineering	Use derived fields such as distance_km; one-hot encoding of categorical variables	Completed (initial); time features planned
4	Exploratory data analysis	Distributions and relationships with traffic, vehicle, city, weather and multiple deliveries	Completed
5	KPI analysis	Average delivery time, distance and rider rating	Completed
6	Predictive modelling	Simple and multiple linear regression	Completed
7	Delivery segmentation	K-Means clustering with K = 5	Completed
8	Optimization	Conceptual objectives and constraints	Framework defined; algorithm future work
9	Model evaluation	R ² , MAE and RMSE on a test set	Completed for linear models; further validation planned
10	Deployment / dashboard	KPI dashboard and delivery-time estimates	Future work
11	Continuous monitoring	Track KPIs and model performance over time	Future work

Table 12: End-to-end project roadmap and current status

15. Python Implementation / Code Snippets

The analysis was carried out in Python in a Google Colab notebook (Logistics_Delivery_Analysis.ipynb), using pandas, NumPy, Matplotlib, seaborn and scikit-learn [5]. The listings below are taken from that notebook. They are shortened to the essential lines, and plotting-format lines and display-only cells are left out. Variable names and values are unchanged.

15.1 Dataset loading

The dataset was read into a pandas DataFrame from the CSV file used in the Colab notebook, and its dimensions were checked with `df.shape`.

```
df = pd.read_csv("/content/zomato_cleaned.csv")

df.shape
```

Listing 1: Loading the dataset

15.2 Data cleaning

A copy of the original dataframe (`df_clean`) was created so the raw data stays unchanged. Missing values in `Delivery_person_Age` and `Delivery_person_Ratings` were then filled with each column's median.

```
df_clean = df.copy()

df_clean["Delivery_person_Age"] = df_clean["Delivery_person_Age"].fillna(
    df_clean["Delivery_person_Age"].median()
)

df_clean["Delivery_person_Ratings"] = df_clean["Delivery_person_Ratings"].fillna(
    df_clean["Delivery_person_Ratings"].median()
)
```

Listing 2: Creating `df_clean` and applying median imputation

15.3 KPI calculation

Each KPI is the mean of the corresponding column in the cleaned dataframe, rounded to two decimals when printed.

```
average_delivery_time = df_clean["Time_taken (min)"].mean()

average_distance = df_clean["distance_km"].mean()

average_rating = df_clean["Delivery_person_Ratings"].mean()

print("Average Delivery Time:", round(average_delivery_time, 2), "minutes")
print("Average Delivery Distance:", round(average_distance, 2), "km")
print("Average Delivery Rating:", round(average_rating, 2))
```

Listing 3: KPI calculation

15.4 Exploratory data analysis

The scatter plot in Figure 2 was drawn with `seaborn`, using `distance_km` on the x-axis and `Time_taken (min)` on the y-axis.

```
sns.scatterplot(
    data=df_clean,
    x="distance_km",
    y="Time_taken (min)"
)
```

Listing 4: Distance versus delivery time scatter plot

Average delivery time by traffic density was calculated by grouping the cleaned data and sorting the means from highest to lowest. The same groupby pattern was used for vehicle type and city.

```
traffic_analysis = df_clean.groupby(
    "Road_traffic_density"
) ["Time_taken (min)"].mean().sort_values(ascending=False)

print(traffic_analysis)
```

Listing 5: Average delivery time by road traffic density

The weather analysis follows the same pattern, grouping by Weather_conditions.

```
weather_time = df_clean.groupby(
    "Weather_conditions"
) ["Time_taken (min)"].mean().sort_values(ascending=False)

print(weather_time)
```

Listing 6: Average delivery time by weather condition

Here the grouping variable is multiple_deliveries, which is numeric (0 to 3).

```
multiple_delivery_time = df_clean.groupby(
    "multiple_deliveries"
) ["Time_taken (min)"].mean()

print(multiple_delivery_time)
```

Listing 7: Average delivery time by number of multiple deliveries

15.5 Simple linear regression

The first block selects the feature (distance_km) and target (Time_taken (min)), splits the data 80/20, fits a LinearRegression model and predicts delivery times for the test set.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

X = df_clean[["distance_km"]]
y = df_clean["Time_taken (min)"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

Listing 8: Simple linear regression: setup, training and prediction

The second block calculates R^2 , MAE and RMSE on the test set. RMSE is the square root of the mean squared error.

```
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("R2 Score:", round(r2, 4))
print("MAE:", round(mae, 2), "minutes")
print("RMSE:", round(rmse, 2), "minutes")
```

Listing 9: Simple linear regression: evaluation

15.6 Multiple linear regression

The 11 selected features are stored in a list, and `pd.get_dummies` with `drop_first = True` converts the categorical variables to numeric columns. This produces the 21 encoded features in `X_encoded`.

```
features = [
    "distance_km",
    "Delivery_person_Age",
    "Delivery_person_Ratings",
    "Vehicle_condition",
    "multiple_deliveries",
    "Weather_conditions",
    "Road_traffic_density",
    "Type_of_order",
    "Type_of_vehicle",
    "Festival",
    "City"
]

X = df_clean[features]
y = df_clean["Time_taken (min)"]

X_encoded = pd.get_dummies(X, drop_first=True)
```

Listing 10: Multiple regression: feature selection and one-hot encoding

The same 80/20 split and `random_state` were used as in the simple model, followed by model training and prediction.

```
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded,
    y,
    test_size=0.2,
    random_state=42
)

multiple_model = LinearRegression()
multiple_model.fit(X_train, y_train)

y_pred_multiple = multiple_model.predict(X_test)
```

Listing 11: Multiple regression: train-test split, training and prediction

The three evaluation metrics are calculated in the same way as for the simple model.

```
r2_multiple = r2_score(y_test, y_pred_multiple)
mae_multiple = mean_absolute_error(y_test, y_pred_multiple)
rmse_multiple = np.sqrt(
    mean_squared_error(y_test, y_pred_multiple)
)
```

Listing 12: Multiple regression: evaluation

Coefficients were extracted from `multiple_model.coef_` and sorted by absolute size to find the most influential features (Table 10).

```
coefficients = pd.DataFrame({
    "Feature": X_encoded.columns,
    "Coefficient": multiple_model.coef_
})

coefficients["Absolute_Coefficient"] = coefficients["Coefficient"].abs()

coefficients = coefficients.sort_values(
    "Absolute_Coefficient",
    ascending=False
)
```

Listing 13: Extracting and ranking regression coefficients

15.7 Regression model comparison

To compare the models, the metrics were stored in lists. The simple model's results (0.1084, 7.07, 8.71) were entered as values from the earlier run, while the multiple model's results come directly from the variables above. A bar chart of the R^2 scores produced Figure 11.

```
model_names = ["Simple Linear Regression", "Multiple Linear Regression"]

r2_scores = [0.1084, r2_multiple]
mae_scores = [7.07, mae_multiple]
rmse_scores = [8.71, rmse_multiple]

plt.bar(model_names, r2_scores)
```

Listing 14: Collecting results and plotting the R^2 comparison

15.8 K-Means clustering

The three clustering features were selected from `df_clean` and standardized with `StandardScaler` so that no single feature dominates because of its scale.

```
cluster_features = df_clean[
    ["distance_km", "Time_taken (min)", "Delivery_person_Ratings"]
]

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(cluster_features)
```

Listing 15: Selecting and standardizing the clustering features

The elbow method fits K-Means for K = 2 to 7 and records the inertia (within-cluster sum of squares) for each value.

```
from sklearn.cluster import KMeans

inertia = []

for k in range(2, 8):
    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)
```

Listing 16: Elbow method

K-Means was then fitted with K = 5, and the resulting cluster label for each delivery was stored in a new Cluster column.

```
kmeans = KMeans(
    n_clusters=5,
    random_state=42,
    n_init=10
)

df_clean["Cluster"] = kmeans.fit_predict(X_scaled)
```

Listing 17: K-Means with K = 5 and cluster assignment

15.9 Cluster summary

The cluster profile in Table 11 was produced by averaging distance, delivery time and rating within each cluster.

```
cluster_summary = df_clean.groupby("Cluster") [
    ["distance_km", "Time_taken (min)", "Delivery_person_Ratings"]
].mean().round(2)
```

Listing 18: Cluster summary

15.10 Optimization framework

The notebook defines the framework as two Python lists, one for objectives and one for constraints, and prints them. It is a conceptual definition only; no solver or routing algorithm is run.

```
# Simple logistics optimization framework

optimization_objectives = [
    "Minimize delivery time",
    "Reduce transportation cost",
    "Improve rider/resource utilization",
    "Prioritize high-delay delivery segments"
]

optimization_constraints = [
    "Limited number of delivery riders",
    "Vehicle availability",
    "Delivery distance",
    "Traffic conditions",
    "Delivery capacity"
]

print("Optimization Objectives:")
for objective in optimization_objectives:
    print("-", objective)

print("\nOptimization Constraints:")
for constraint in optimization_constraints:
    print("-", constraint)
```

Listing 19: Optimization objectives and constraints

16. Key Findings

- The dataset contains 38,964 records.
- Average delivery time was 26.58 minutes.
- Average delivery distance was 9.77 km.
- Average delivery-person rating was 4.63/5.
- Distance-only regression achieved $R^2 = 0.1084$, so distance alone has limited explanatory power.
- Multiple regression achieved $R^2 = 0.5903$.
- Multiple regression achieved MAE = 4.69 minutes and RMSE = 5.90 minutes, compared with 7.07 and 8.71 minutes for the distance-only model.
- Higher numbers of parallel deliveries, Jam traffic, and Fog or Cloudy weather were associated with longer average delivery times in the EDA.
- Five delivery segments were identified using K-Means.
- Cluster 1 had the highest average delivery time at 40.89 minutes.
- Cluster 2 had the fastest average delivery time at 16.80 minutes.
- Cluster 4 had the lowest average rating at 4.01.

- Cluster 3 showed that long distance does not necessarily mean slow delivery (15.42 km in 22.51 minutes).
- Delivery performance is associated with multiple operational factors rather than distance alone.

17. Recommendations

The following recommendations are based on the analysis. None of them has been implemented or tested in this project.

- Prioritize high-delay segments (especially Clusters 1 and 4) for operational intervention.
- Consider traffic conditions during rider allocation and dispatching.
- Monitor multiple-delivery workloads, because additional deliveries are associated with longer delivery times.
- Use clustering to identify operationally different delivery segments.
- Use predictive models to estimate delivery time before dispatch.
- Improve rider and resource allocation using predicted workload.
- Consider weather and festival conditions in planning.
- Integrate route optimization and real-time traffic data in future versions.

18. Expected Outcomes

18.1 Demonstrated in this project

- Baseline KPIs for delivery time, distance and rider rating.
- A multiple regression model that explains about 59% of the variation in delivery time, with an average error (MAE) of 4.69 minutes on the test set.
- Five delivery segments, including two high-delay segments (Clusters 1 and 4) that can be targeted.
- A conceptual optimization framework with defined objectives and constraints.

18.2 Expected future impact

The outcomes below are expected benefits that depend on further work. They have not been demonstrated in this project.

- Better delivery-time prediction through additional features and further validation.
- Improved rider allocation based on predicted workload.
- Easier identification of high-delay segments through regular clustering.
- Improved route planning once a routing model is implemented.
- Reduced operational inefficiency.
- Improved customer experience through more reliable delivery estimates.
- Stronger data-driven decision-making across the operations team.

19. Conclusion

This project analysed 38,964 food-delivery records to study what influences delivery time. The baseline KPIs were an average delivery time of 26.58 minutes, an average distance of 9.77 km and an average rider rating of 4.63/5.

A simple regression using distance alone gave $R^2 = 0.1084$, which shows that distance has limited explanatory power. A multiple regression model that added rider, vehicle, weather, traffic, festival and city variables reached $R^2 = 0.5903$. MAE improved from 7.07 to 4.69 minutes and RMSE from 8.71 to 5.90 minutes. These results show associations in the data, not proof of cause and effect. K-Means clustering identified five delivery segments. In particular, Cluster 3 (15.42 km in 22.51 minutes) was faster than Cluster 0 (6.14 km in 28.03 minutes), which confirms that distance alone does not fully explain delivery performance.

Overall, the project shows how data science can support predictive logistics and provide a base for future optimization. The limits are clear: the models are linear, variables such as time of day and preparation time are not yet used, some categories (Semi-Urban, Festival) are small, and the optimization component is conceptual only. The models are an initial analysis and are not production-ready. The next steps are richer features, further validation and an implemented routing and dispatch model.

20. References

- [1] Arvis, J.-F., Ojala, L., Shepherd, B., Ulybina, D., & Wiederer, C. (2023). Connecting to Compete 2023: Trade Logistics in an Uncertain Global Economy. The Logistics Performance Index and Its Indicators. Washington, DC: The World Bank.
https://lpi.worldbank.org/sites/default/files/2023-04/LPI_2023_report_with_layout.pdf
- [2] allenborochin. zomato_delivery_EDA [Dataset]. Hugging Face.
https://huggingface.co/datasets/allenborochin/zomato_delivery_EDA
- [3] Jazemi, R., Alidadiani, E., Ahn, K., & Jang, J. (2023). A review of literature on vehicle routing problems of last-mile delivery in urban areas. Applied Sciences, 13(24), 13015.
<https://doi.org/10.3390/app132413015>
- [4] Auad, R., Lagos, F., & Lagos, T. Data-driven optimization for meal delivery: A reinforcement learning approach for order-courier assignment and routing at Meituan. Transportation Science (published online). <https://doi.org/10.1287/trsc.2025.0129>
- [5] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.